

Upskill Advisor – Extended Architecture with LLM & MongoDB

The Upskill Advisor system recommends courses based on roles and skills using both **rule-based JSON mappings** and an **LLM-powered RAG pipeline**. MongoDB is used as the **vector store** to manage embeddings and enable semantic retrieval.

Extended System Architecture

1. React Frontend

- UI for role & skill input.
- Displays both JSON-based and LLM+RAG recommendations.
- Allows natural language queries (e.g., “I’m a QA looking to move into GenAI QA, suggest a roadmap”).

2. Python Backend (FastAPI)

- Exposes REST API endpoints for recommendations.
- Runs **dual pipeline**:
 - **Baseline engine** → Matches from courses.json and jds.json.
 - **RAG engine** → Embeds queries, retrieves top results from MongoDB Vector Store, augments context, and queries LLM.
- Services include:
 - **Retriever Service** → MongoDB vector search.
 - **LLM Service** → Ollama Phi-3.
 - **Evaluator Service** → Metrics for accuracy, diversity, latency.

3. Data Sources

- **courses.json** → curated course metadata (title, provider, URL).
- **jds.json** → role-to-skill mappings.
- **MongoDB Vector Store** → stores embeddings of courses, skills, and job descriptions for semantic retrieval.

4. Embedding Model (MiniLM)

- Generates compact, efficient embeddings for:
 - User input (role + skills).
 - Course descriptions.
 - Job requirements.

- Stored/retrieved from MongoDB for similarity search.

5. LLM Layer (Ollama Phi-3)

- Acts as reasoning engine.
 - Takes retrieved context + user input.
 - Generates personalized course recommendations and justifications.
 - Example: “Since you are a QA Engineer with manual testing experience, I suggest transitioning via Automation → AI in QA → GenAI Testing Tools.”
-

Data Flow with RAG + MongoDB

1. **Input** → User submits role & skills in frontend.
 2. **Embedding** → Backend generates embeddings via MiniLM.
 3. **Retrieval** → Similar vectors fetched from MongoDB Vector Store.
 4. **Augmentation** → Retrieved context is injected into the LLM prompt.
 5. **Generation** → Ollama Phi-3 produces refined recommendations.
 6. **Response** → Combined JSON-based + AI-driven recommendations sent to frontend.
 7. **Display** → React shows top-3 structured + contextual recommendations.
-

Technology Stack

- **Frontend:** React.js
 - **Backend:** FastAPI (Python)
 - **LLM:** Ollama Phi-3
 - **Embeddings:** MiniLM (SentenceTransformers)
 - **Vector Store:** MongoDB (Atlas Vector Search)
 - **Data Sources:** courses.json, jds.json, MongoDB embeddings
-

Benefits of Using MongoDB Vector Store

- **Single data backend** → Combines structured JSON-like storage + vector embeddings in one DB.
- **Scalable** → Handles growing courses and job descriptions easily.
- **Flexible queries** → Mix structured filters (role, difficulty) with semantic search.